



Universidad Autónoma de Campeche

Facultad de Ingeniería

Ingeniería en Tecnología de Software

29 de Octubre del 2026

Materia:
Concurrencia y Paralelismo.

Actividad:
Reporte: Practica 3- Servidor Concurrente.

Docente:
Sergio Andres Noh Puch.

Alumno:
Pedro Alejandro Chávez Rivero.

1. ¿Qué cambió respecto al código original?

En el código original, el cliente únicamente enviaba una cadena estática y terminaba inmediatamente la ejecución del método main, provocando que el bloque try-with-resources cerrara el socket tras ese primer envío sin leer la respuesta del servidor. En la versión modificada se incorporó un objeto Scanner dentro de un ciclo while (true) que captura múltiples mensajes desde la consola y lee la respuesta del servidor mediante entrada.readLine() en cada iteración. Además, en ambas clases se implementó una condición de salida limpia evaluando la palabra clave "salir", y en el método handleRequest del servidor se incluyó explícitamente el clientSocket dentro del bloque try-with-resources para garantizar que el socket y el hilo de atención se liberen ordenadamente al finalizar la sesión.

2. ¿Por qué la conexión ahora se mantiene viva?

La conexión se mantiene viva porque el protocolo TCP establece un canal de comunicación bidireccional persistente mientras ninguno de los extremos invoque el método close() sobre su socket. Al colocar las operaciones de envío (println) y lectura (readLine) dentro de ciclos while tanto en el cliente como en el hilo del servidor, el flujo de ejecución permanece dentro del bloque try-with-resources sin destruir el socket tras el primer mensaje, reutilizando los mismos flujos de entrada y salida durante toda la conversación.

3. ¿Qué pasa internamente cuando el cliente deja de enviar datos?

Si el cliente sigue conectado pero el usuario no escribe nada, el hilo correspondiente en el servidor permanece bloqueado de forma síncrona en el método in.readLine(); el sistema operativo pone a ese hilo en estado de espera por Entrada/Salida (I/O) sin consumir CPU hasta que lleguen nuevos datos seguidos de un salto de línea. Por otro lado, cuando el cliente escribe "salir" o cierra su proceso, su socket se cierra y envía un segmento TCP FIN. Al recibirlo, el servidor detecta el fin del flujo (EOF), haciendo que in.readLine() devuelva null (o se ejecute el break por palabra clave), lo que rompe el ciclo while, cierra los flujos locales y finaliza el hilo sin dejar recursos colgados.